# Course 2 Master Revision: Building RAG Applications

Last corrected: 8 August 2026

## What the course established

You completed the introductory RAG flow in LangChain and LlamaIndex. The labs prove framework familiarity, not yet a production RAG implementation in the main package. Courses 3-4 and the portfolio bridge must add real vector retrieval, measured quality, stable citations, security filtering, and service reliability.

## 1. RAG mental model

RAG is two systems joined together:

1. Retrieval selects candidate evidence from an external corpus.
2. Generation uses that evidence to answer or refuse.

RAG can improve freshness, access to private data, and attribution. It does not force the model to use context correctly and does not eliminate unsupported answers.

Diagnose stages separately:

- Parsing: did the source content/structure survive ingestion?
- Chunking: is the answerable unit represented coherently?
- Indexing: were vectors/text/metadata written correctly?
- Retrieval: did relevant evidence rank highly under the correct access filters?
- Context assembly: was evidence kept within budget and positioned well?
- Generation: did the model answer faithfully and cite support?
- Validation: were schema, citation, safety, and business rules enforced?

## 2. Ingestion and provenance

Recommended ingestion stages:

```
source -> authorization -> parse -> normalize -> deduplicate -> chunk -> embed -> index -> verify
```

Preserve stable document/chunk IDs plus source URL, page, section, company, reporting period, access scope, parser version, embedding version, and ingestion timestamp.

Parsing financial PDFs requires special care for tables, reading order, headers/footers, scanned pages, and units. Prefer structure-aware parsers when needed; use vision/OCR only when layout or scans justify the extra cost and evaluation burden.

## 3. Chunking

- Fixed-size splitting is predictable but may cut semantic units.
- Recursive splitting uses a hierarchy of separators and is a practical baseline.
- Structure-aware splitting respects headings, pages, tables, or sections.
- Semantic splitting can group topic-consistent text but adds model cost/complexity.
- Parent-child retrieval indexes small children and returns larger parents.

Tune chunk size/overlap on an evaluation set. Larger chunks are not automatically better just because a model has a long context window.

Project correction: the current `split_document` is paragraph grouping, not a complete recursive splitter; it ignores `chunk_overlap` and can emit an oversized paragraph.

## 4. Embeddings and similarity

- Cosine similarity compares direction and ignores magnitude.
- Dot product includes magnitude and equals cosine for normalized vectors.
- Euclidean/L2 measures geometric distance.

Follow the embedding model/index recommendation. Store the embedding model/version and re-index deliberately when it changes.

Vector search returns semantically nearby content, not verified facts. Approximate indexes trade some recall for latency/scale. Metadata filters are part of security, not an optional UI feature.

## 5. Retrieval strategies

Dense search helps semantic matches. Sparse/BM25 helps exact terms, identifiers, figures, and names. Financial and technical corpora often benefit from hybrid retrieval.

Typical measured pipeline:

```
dense + sparse candidates -> reciprocal-rank fusion -> filters -> reranker -> context budget
```

Rerankers can improve ordering because they jointly inspect query and candidate, but they add cost and latency. Keep them only if evaluation shows a useful gain.

Top-k trades recall against precision, cost, and lost-in-the-middle risk. Query rewriting, multi-query retrieval, hypothetical-document techniques, and self-query filters should be introduced only for a measured failure mode.

## 6. Context combination and conversational state

"Stuff" concatenates selected chunks into one prompt and is a strong simple baseline when the context fits. It is not universally used in a fixed percentage of production systems.

Map-reduce, refine, and tree/compact synthesis can help long summarization or staged aggregation, but they add calls, latency, and new failure modes. Choose from workload/eval evidence.

Conversation history is application state injected into future requests; the model itself is stateless between calls. Long histories increase cost and can pollute retrieval. Condense follow-up questions or retain structured state, and evaluate reference resolution.

## 7. Citations and grounding

Returning retrieved `Document` objects shows what was selected, not that an answer is supported.

A useful citation pipeline returns stable source/page/chunk identifiers and checks whether each claim is entailed by the cited span. For extracted metrics, verify company, reporting period, metric name, value, currency/unit, and scale.

Prompts such as "use only this context" help behavior but are not enforcement. Refusal accuracy must be tested when no supporting evidence exists.

## 8. Evaluation

Build a versioned set of questions, expected answers/claims, relevant source IDs, expected refusals, and tags.

Retrieval metrics:

- Recall@k and precision@k.
- Mean reciprocal rank.

- nDCG for graded relevance.
- Access/filter correctness.

Generation/application metrics:

- Answer correctness.
- Citation correctness/support.
- Faithfulness/groundedness.
- Refusal accuracy.
- Schema success, latency, token/cost, error and retry rates.

Use deterministic checks for exact facts, IDs, units, and citation spans. LLM-as-judge can help with nuanced responses, but calibrate it to human labels and expect bias, variance, and model/version drift.

## 9. Security failure modes

Treat retrieved content as untrusted data.

- Indirect prompt injection can instruct the model to ignore policy or call tools.
- Inadequate tenant/metadata filtering can leak private documents.
- Improper output handling can pass model-generated commands or markup into downstream systems.
- Poisoned sources can manipulate retrieval and answers.
- Unbounded retrieval/context/model use can cause denial of wallet/service.

Mitigations include data/source controls, access filters before retrieval, content delimiting, least-privilege tools, application-side authorization, output encoding/validation, budgets, provenance, and adversarial evals.

## 10. LangChain and LlamaIndex mapping

| Concern | LangChain | LlamaIndex |
|---|---|---|
| Document text | `Document.page_content` | `Document.text` |
| Chunk abstraction | often `Document` chunks | `TextNode`/Node |
| Composition | runnables/LCEL and integrations | indexes, retrievers, query/chat engines |
| Retrieval | retriever interfaces/vector-store adapters | `index.as_retriever()` and retrievers |
| Generation | prompt/model/parser or retrieval chains | response synthesizer/query engine |
| Conversation | message history/workflow state | chat engine/memory patterns |

Objects are not interchangeable by changing imports. Convert explicitly and test metadata/relationships.

Frameworks accelerate composition, but production boundaries should hide framework-specific APIs from domain logic.

## 11. Legacy versus durable knowledge

Course labs may show older classes such as `RetrievalQA`, `LLMChain`, `ConversationalRetrievalChain`, or older configuration patterns. Understand what they do, but check current official documentation before new implementation.

Durable knowledge:

- Typed boundaries.
- Explicit dataflow.
- Retrieval/generation separation.
- Provenance and access filters.
- Evaluation-driven configuration.
- Bounded failures and observable execution.

## 12. Portfolio bridge: next implementation

The next RAG slice belongs in `src/ai_engineering_lab`, not only `coursera_notes`:

1. Define document/chunk records with stable IDs and provenance.
2. Fix and test the splitter.
3. Ingest two or three public earnings documents.
4. Add embeddings and a persistent vector index.
5. Return ranked chunks/scores without generation.
6. Create 15 retrieval questions with relevant chunk IDs.
7. Report recall@k and MRR.
8. Add sparse retrieval/fusion and compare.
9. Add generation, refusals, and stable citations.
10. Test claim-to-citation support.

## 13. Interview drills

1. A RAG answer is wrong. How do you isolate parsing, retrieval, and generation failure?
2. Why might hybrid retrieval outperform dense search for earnings documents?
3. How do chunk size, overlap, top-k, and reranking affect quality/latency/cost?
4. What does recall@k measure, and why is answer faithfulness separate?
5. Why are returned source documents not proof of grounding?
6. How do metadata filters become a security control?
7. When would long context be simpler than RAG?
8. What would you measure before keeping a reranker?

## 14. One-minute project answer

"I completed introductory RAG implementations in LangChain and LlamaIndex, but I keep those separate from what the main application actually supports. The current package has chunk/prompt utilities, not vector retrieval. My next slice adds stable chunk provenance, persistent embeddings, ranked retrieval, and a versioned set where I measure recall@k and MRR before generation. Then I add hybrid retrieval, citations, refusal tests, and claim-support checks. That lets me distinguish a retrieval miss from a generator or citation failure instead of tuning prompts blindly."